VIT BHOPAL

FUNDAMENTALS IN AI AND ML

(CSA2001)

Shrestha Sarkar
25BAC10015
BTechECE(AI and Cybernetics)

Smart Network Bandwidth Monitor

PROBLEM STATEMENT

Users often experience slow internet but cannot identify which applications are consuming bandwidth. There is no simple, real-time, and lightweight tool to monitor network usage and visualize upload and download speeds. This project aims to develop a Smart Network Bandwidth Monitor to track real-time internet usage and analyze network performance efficiently.

SCOPE OF THE PROJECT

The Smart Network Bandwidth Monitor is designed to provide real-time monitoring and analysis of internet usage on a system. The project focuses on tracking upload and download speeds, visualizing bandwidth consumption, and helping users identify network performance issues. This tool aims to offer a lightweight, user-friendly solution for monitoring network activity and improving overall internet usage efficiency.

FUNCTIONAL REQUIREMENTS

1. **Real-Time Network Monitoring**
   The system should monitor and display upload and download speeds continuously in real time.
2. **Bandwidth Calculation**
   The application should calculate internet speed based on bytes sent and received over a period of time.
3. **Data Visualization**
   The system should generate graphs to represent upload and download speed trends for better understanding.
4. **User Interface Display**
   The application should provide a simple dashboard displaying network usage clearly.

5. **Continuous Data Collection**
   The system should continuously collect and update network usage data at regular intervals.

NON FUNCTIONAL REQUIREMENTS

1. **Performance**
   The system should operate efficiently without significantly affecting system performance.
2. **Usability**
   The interface should be simple, easy to use, and understandable for all users.
3. **Reliability**
   The system should run continuously and provide accurate network monitoring.
4. **Portability**
   The application should run on different operating systems such as Windows, Linux, and macOS.
5. **Maintainability**
   The code should be modular and easy to update for future enhancements.

SYSTEM ARCHITECTURE

Overview

The Smart Network Bandwidth Monitor follows a simple layered architecture where the system collects network data from the operating system, processes it using Python, and displays the results using a graphical interface. The architecture ensures real-time monitoring, efficient processing, and easy visualization of network usage.

The system consists of four main components:

- Data Collection Layer
- Processing Layer
- Visualization Layer

- User Interface Layer

SYSTEM ARCHITECTURE COMPONENTS

1. Data Collection Layer

This is the first layer of the system where network data is collected from the computer.

Functions

- Collect bytes sent and received
- Monitor network activity
- Fetch system network statistics

Tools Used

- Python
- psutil library

The `psutil` library interacts with the operating system and retrieves real-time network usage data.

2. Processing Layer

This layer processes the collected data and calculates network speed.

Functions

- Calculate upload speed
- Calculate download speed
- Convert bytes to KB/s or MB/s
- Store processed data temporarily

Working

The system calculates:

Upload Speed = Bytes Sent Difference / Time

Download Speed = Bytes Received Difference / Time

This allows the system to calculate real-time internet speed.

3. Visualization Layer

This layer converts processed data into graphs and charts.

Functions

- Generate real-time graphs
- Show bandwidth usage trends
- Update graphs dynamically

Tools Used

- Matplotlib
- Pandas

The system continuously updates graphs showing upload and download speeds.

4. User Interface Layer

This is the front-end layer where users interact with the system.

Functions

- Display upload speed
- Display download speed
- Show real-time graphs
- Provide dashboard view

Tools Used

- Streamlit

Streamlit creates a simple web-based dashboard for monitoring.

SYSTEM ARCHITECTURE FLOW

Operating System

↓

Data Collection (psutil)

↓

Processing Layer (Python)

↓

Data Storage (Pandas)

↓

Visualization (Matplotlib)

↓

User Interface (Streamlit Dashboard)

ARCHITECTURE DIAGRAM

```
+----------------------+
|   User Interface    |
|     (Streamlit)     |
+----------↑-----------+
          |
```

```
+----------|-----------+
|   Visualization     |
|    (Matplotlib)     |
+----------↑-----------+
          |
+----------|-----------+
|   Processing Layer   |
|     (Python)        |
+----------↑-----------+
          |
+----------|-----------+
|  Data Collection     |
|     (psutil)        |
+----------↑-----------+
          |
+----------------------+
|   Operating System   |
+----------------------+
```

USE CASE DIAGRAM(TEXTUAL)

```
                +---------------------------+
                |   Smart Network Monitor   |
                +---------------------------+

      User
       |
       |
       |------------------> Start Monitoring
       |
       |------------------> View Upload Speed
       |
       |------------------> View Download Speed
       |
       |------------------> View Real-Time Graph
       |
       |------------------> Analyze Bandwidth Usage
       |
       |------------------> Stop Monitoring
```

WORKFLOW/ACTIVITY DIAGRAM

```
Start
  ↓
User Runs Network Monitor
  ↓
System Initializes Libraries
  ↓
Collect Initial Network Data
  ↓
Start Monitoring Loop
  ↓
Collect Network Usage Data
  ↓
Calculate Upload Speed
  ↓
Calculate Download Speed
  ↓
Store Data Temporarily
  ↓
Generate Graphs
  ↓
Display Results on Dashboard
```

```
  ↓
Update Data Continuously
  ↓
User Stops Monitoring?
     ↓            ↓
    No           Yes
     ↓            ↓
Continue Loop    Stop Monitoring
     ↓            ↓
     ←-------------
           ↓
         End
```

SEQUENCE DIAGRAM

```
User          UI (Streamlit)       Network Monitor        System (psutil)        Graph Modu
 |                 |                     |                     |                    |
 |----Start App---->|                     |                     |                    |
 |                 |----Initialize------>|                     |                    |
 |                 |                     |----Request Data---->|                    |
 |                 |                     |<---Network Data-----|                    |
 |                 |                     |----Process Data-----|                    |
 |                 |<---Display Speed----|                     |                    |
 |                 |                     |----Send Data--------------------------->|
 |                 |<---Display Graph--------------------------------------------|
 |                 |                     |                     |                    |
 |<---View Output---|                     |                     |                    |
 |                 |                     |                     |                    |
 |----Stop----------|                     |                     |                    |
 |                 |----Terminate------->|                     |                    |
```

ER DIAGRAM

```
+----------------------+
|      User            |
+----------------------+
| - start()            |
| - stop()             |
+----------+-----------+
           |
           v

+----------------------+
|  NetworkMonitor      |
+----------------------+
| - upload_speed       |
| - download_speed     |
| - data               |
+----------------------+
| + get_network_data() |
| + calculate_speed()  |
| + update_data()      |
+----------+-----------+
           |
```

```
            v

+----------------------+
|  DataVisualizer      |
+----------------------+
| - graph_data         |
+----------------------+
| + generate_graph()   |
| + update_graph()     |
+----------+-----------+
           |
           v

+----------------------+
| StreamlitInterface   |
+----------------------+
| - display_data       |
+----------------------+
| + show_dashboard()   |
| + update_display()   |
+----------------------+
```

DESIGN DECISION AND RATIONAL

Design Decisions (Point-wise)

1. Python as Programming Language

Decision: Use Python for development
Rationale: Easy to implement, strong library support, and suitable for real-time monitoring.

2. Use of psutil Library

Decision: Use psutil for collecting network data
Rationale: Provides accurate and real-time system network statistics.

3. Streamlit for User Interface

Decision: Use Streamlit for dashboard creation
Rationale: Simple UI development and real-time display capability.

4. Real-Time Monitoring

Decision: Implement continuous monitoring
Rationale: Helps users detect network issues instantly.

5. Data Visualization using Matplotlib

Decision: Use Matplotlib for graphs
Rationale: Simple and effective visualization of bandwidth usage.

6. Modular Architecture

Decision: Separate data collection, processing, and visualization
Rationale: Improves maintainability and scalability.

7. Lightweight Design

Decision: Keep system simple without database
Rationale: Faster performance and easy implementation.

IMPLEMENTATION DETAILS

- Python used for development
- psutil for network data collection
- Calculate upload and download speed
- Pandas for temporary data storage
- Matplotlib for graph visualization
- Streamlit for user interface
- Real-time monitoring using loop
- Display speed and graphs continuously

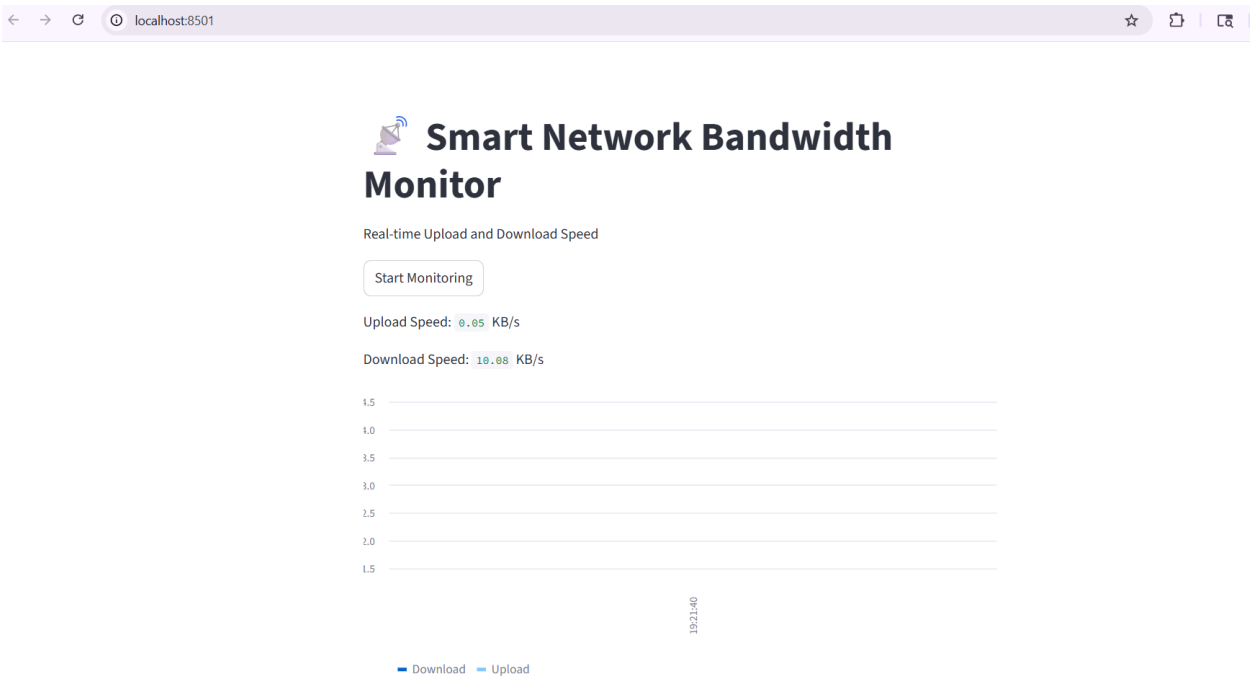
SOURCE CODE

> Users > SHRESTHA SARKAR > OneDrive > Desktop > network_monitor.py > ...

```python
import streamlit as st
import psutil
import pandas as pd
import time
from datetime import datetime

st.title("📡 Smart Network Bandwidth Monitor")

st.write("Real-time Upload and Download Speed")

# Create dataframe
data = pd.DataFrame(columns=["Time", "Upload", "Download"])

# Get initial values
old_value = psutil.net_io_counters()

# Button to start monitoring
if st.button("Start Monitoring"):

    for i in range(50):   # runs 50 times

        time.sleep(1)

        new_value = psutil.net_io_counters()

        upload = (new_value.bytes_sent - old_value.bytes_sent) / 1024
        download = (new_value.bytes_recv - old_value.bytes_recv) / 1024

        old_value = new_value

        current_time = datetime.now().strftime("%H:%M:%S")

        new_row = {
            "Time": current_time,
            "Upload": upload,
```

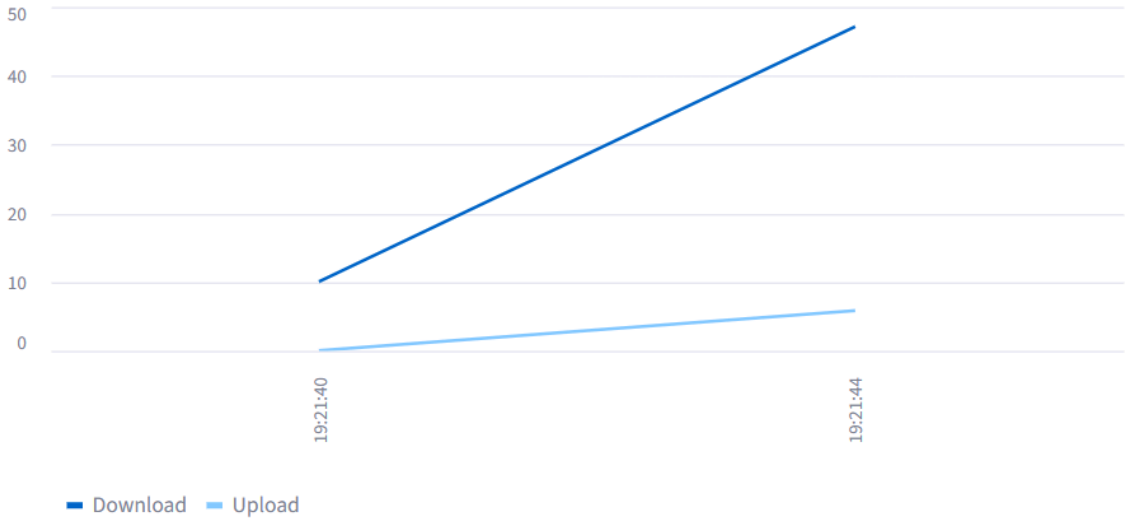
```
            "Download": download
        }

        data = pd.concat([data, pd.DataFrame([new_row])], ignore_index=True)

        st.write("Upload Speed:", round(upload,2), "KB/s")
        st.write("Download Speed:", round(download,2), "KB/s")

        st.line_chart(data.set_index("Time"))

```

OUTPUT



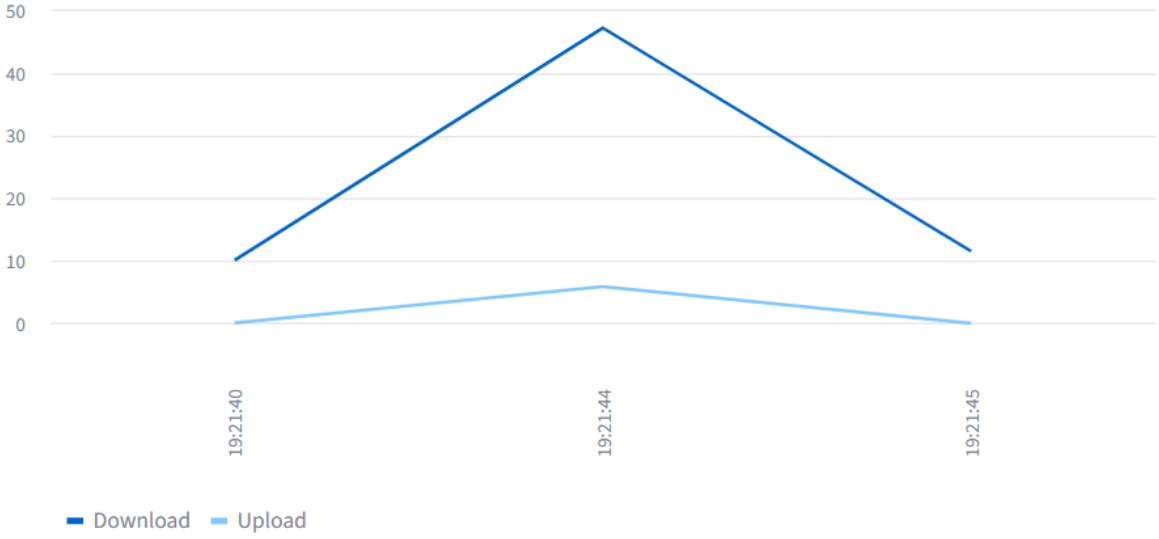
Upload Speed: `5.86` KB/s

Download Speed: `47.15` KB/s



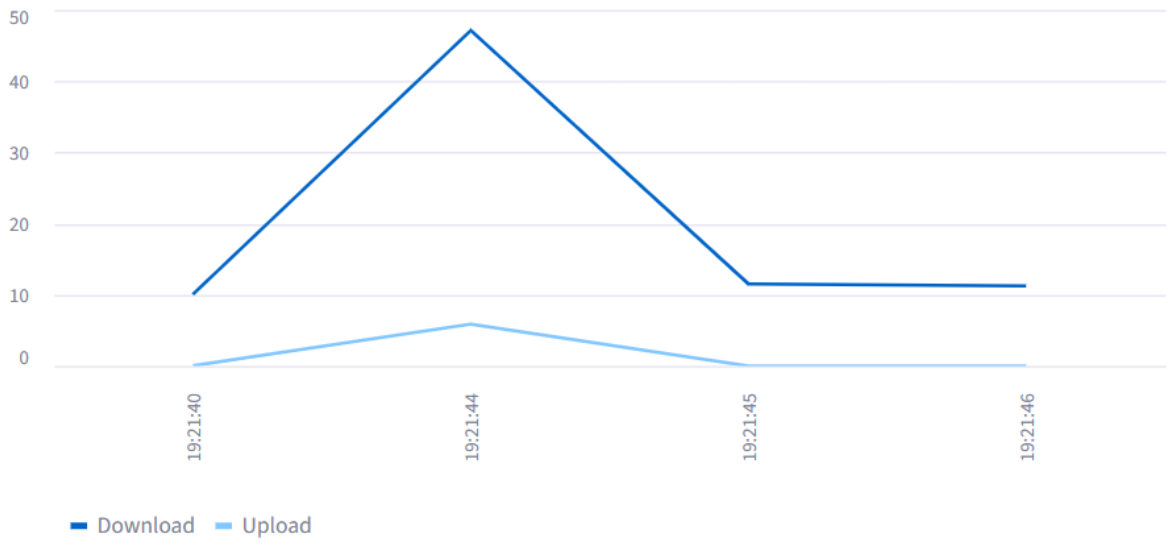
Upload Speed: `0.0` KB/s

Download Speed: `11.53` KB/s



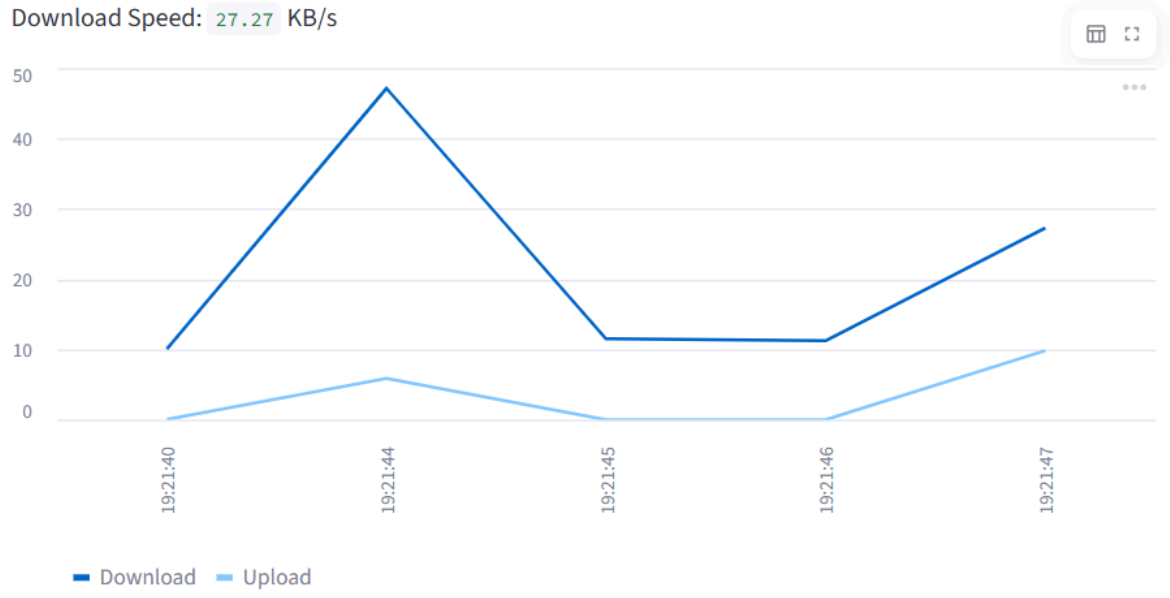
Upload Speed: `0.0` KB/s

Download Speed: `11.25` KB/s



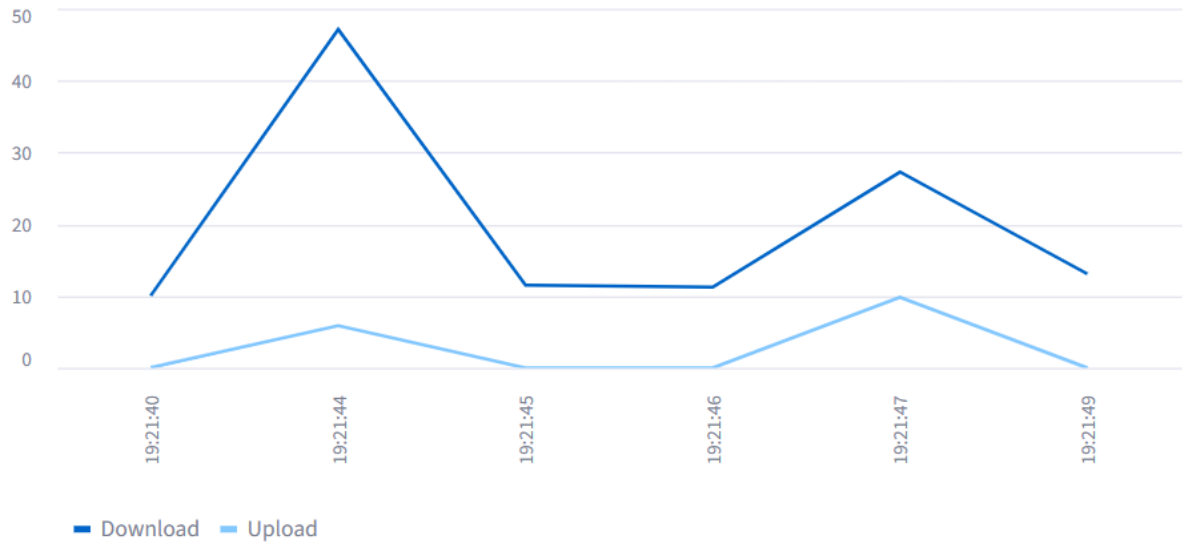
Upload Speed: `9.84` KB/s

Download Speed: `27.27` KB/s



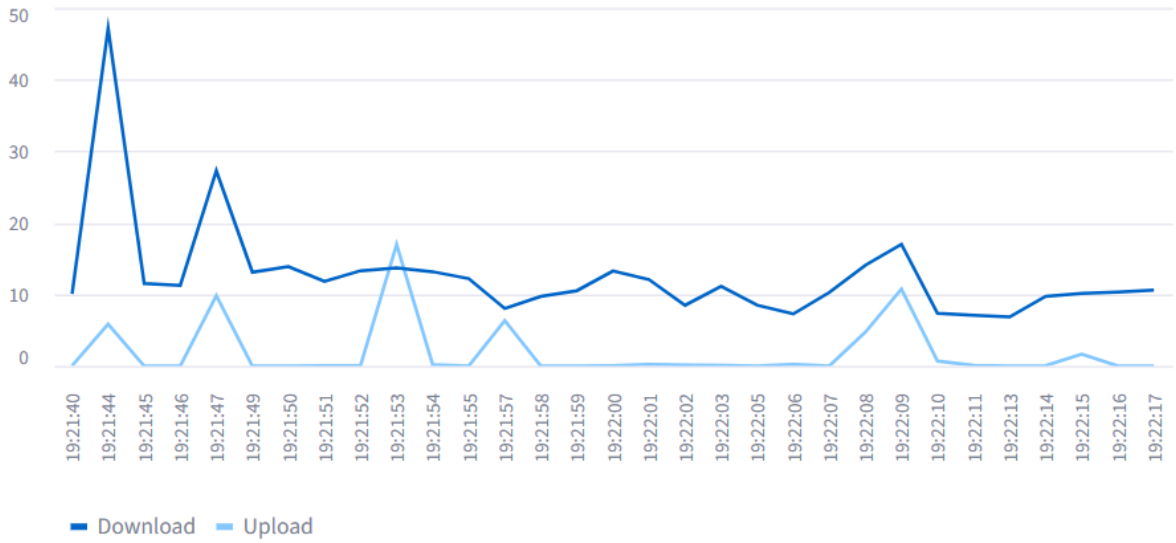
Upload Speed: `0.0` KB/s

Download Speed: `13.1` KB/s



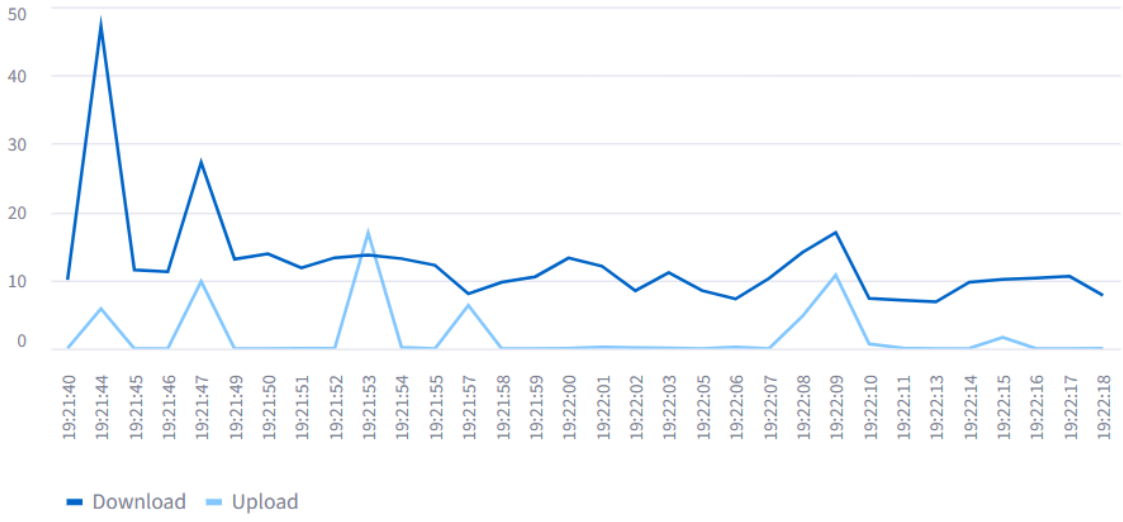
Upload Speed: `0.0` KB/s

Download Speed: `10.59` KB/s



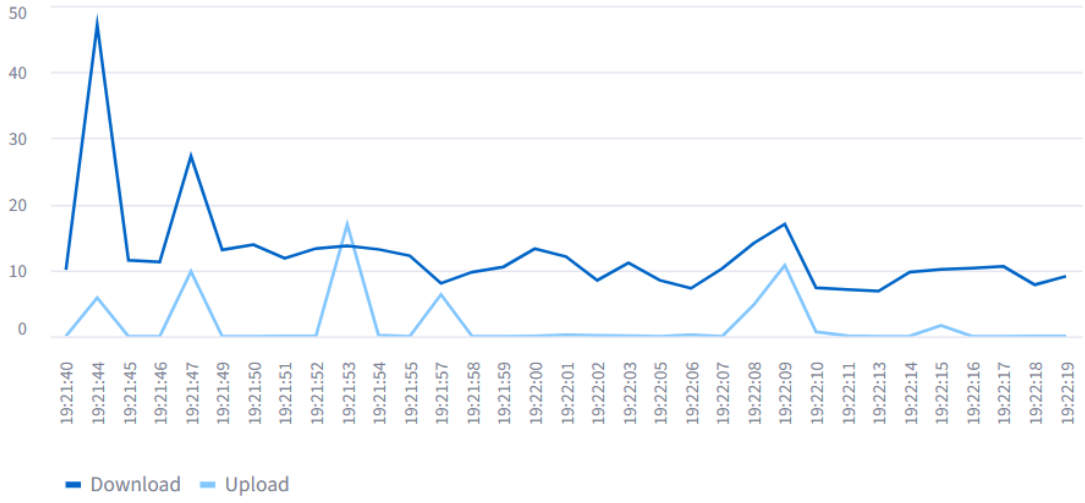
Upload Speed: `0.05` KB/s

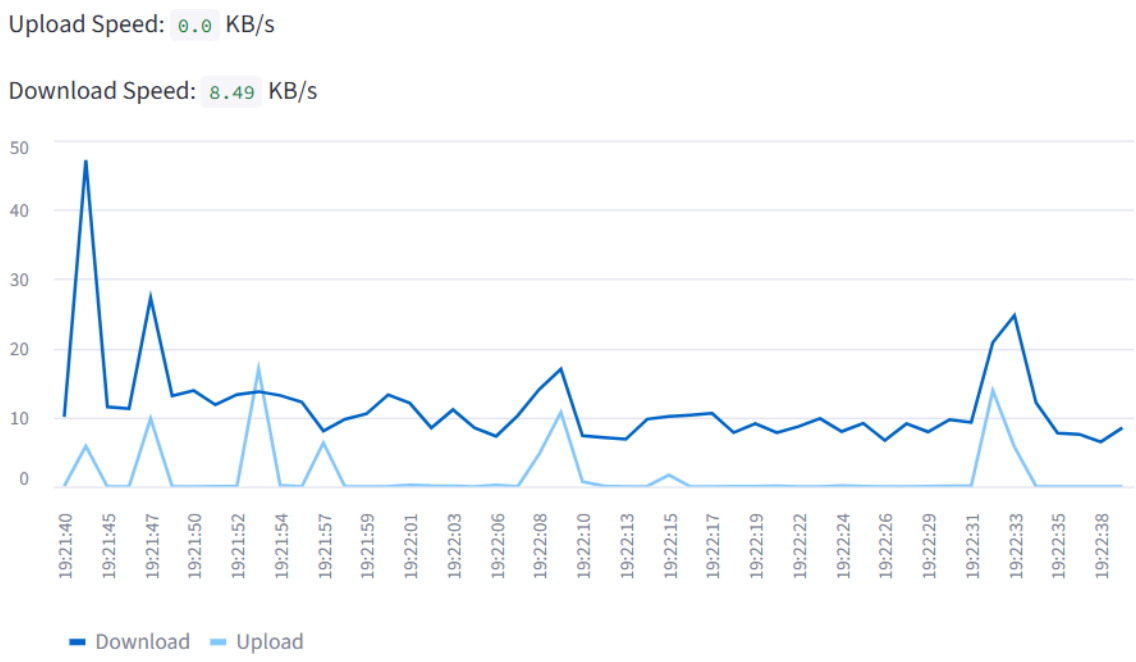
Download Speed: `7.81` KB/s



Upload Speed: `0.05` KB/s

Download Speed: `9.1` KB/s





TESTING APPROACH

- **Unit Testing** — Tested individual modules like data collection and speed calculation
- **Integration Testing** — Verified interaction between monitoring, processing, and UI
- **Functional Testing** — Checked real-time upload and download speed display
- **Performance Testing** — Ensured system runs smoothly without slowing the computer
- **User Interface Testing** — Verified dashboard and graphs display correctly
- **Real-Time Testing** — Tested monitoring with active internet usage
- **Error Testing** — Checked behavior during no internet or low connectivity

CHALLENGES FACED

- Difficulty in collecting real-time network data
- Handling continuous data updates efficiently
- Calculating accurate upload and download speeds
- Managing real-time graph updates
- Ensuring system performance is not affected
- Integrating multiple Python libraries
- Handling no-internet or low-connectivity situations
- Creating a simple and user-friendly interface

LEARNINGS AND KEY TAKEAWAYS

- Mastered file handling (open, read, write, append)
- Understood persistent storage using text files
- Implemented basic user authentication
- Learned menu-driven program design
- Explored timestamp logging with datetime
- Realized importance of input validation & password security
- Gained confidence in building complete console apps from scratch

FUTURE ENHANCEMENTS

- App-wise bandwidth usage monitoring
- Daily and weekly usage reports
- Data usage alerts and notifications
- Bandwidth usage prediction
- Export data to CSV or Excel
- Advanced analytics dashboard
- Multi-device network monitoring
- Mobile-friendly interface

REFERENCES

Python Official Documentation
https://www.python.org/doc/

Psutil Documentation
https://psutil.readthedocs.io/

Streamlit Documentation
https://docs.streamlit.io/

Pandas Documentation
https://pandas.pydata.org/docs/

Matplotlib Documentation
https://matplotlib.org/stable/index.html

Python Networking Concepts
https://realpython.com/python-networking/

Streamlit Tutorial
https://docs.streamlit.io/library/get-started

Psutil GitHub Repository
https://github.com/giampaolo/psutil